

****kwargs: When Customers Give Special Instructions**

Now, a customer says, "Extra cheese on pizza, spicy pasta, less mayo on burger." ****kwargs** handles these named instructions.

Code:

```
def make_food_order_with_instructions(**kwargs):  
    print("Customer's special instructions:")  
    for item, instruction  
    in kwargs.items():  
        print(f"{item}: {instruction}")
```

Customer's order

```
make_food_order_with_instructions( pizza="extra cheese", pasta="spicy", burger="less mayo" )
```

Output:

Customer's special instructions: pizza: extra cheese pasta: spicy burger: less mayo

Explanation:

****kwargs** is another basket, but for named items. Each food item is paired with its instruction, like `pizza="extra cheese"`.

অবশ্যই! নিচে Operator Overloading এর বাংলা অনুবাদ দিলাম তোমার মূল লেখার সাথে মিল রেখে:

অপারেটর ওভারলোডিং (Operator Overloading):

অপারেটর ওভারলোডিং অর্থ হলো অপারেটরগুলোকে (যেমন $+$, $-$, $*$, $/$, $==$ ইত্যাদি) এমনভাবে ব্যবহার করার সুযোগ দেওয়া যাতে তারা কোন ক্লাসের জন্য নিজস্বভাবে কাজ করে। অর্থাৎ, আমরা সাধারণ অপারেটরগুলোর আচরণ পুনঃসংজ্ঞায়িত (redefine) করতে পারি যাতে সেগুলো আমাদের তৈরি অবজেক্টের জন্য বিশেষভাবে কাজ করে।

বাস্তব উদাহরণ:

ধরো তোমার কাছে একটি "Point" নামের ক্লাস আছে যেটা দিয়ে 2D পয়েন্ট (x, y) তৈরি করা হয়। এখন তুমি চাও দুইটা পয়েন্ট যোগ করতে (+) যাতে তাদের x ও y আলাদাভাবে যোগ হয়। সাধারণত + অপারেটর এটা বুঝতে পারে না, কিন্তু অপারেটর ওভারলোডিং এর মাধ্যমে এটা করা যায়।

অংশ (Part)	ব্যাখ্যা (Explanation)
self	বর্তমান অবজেক্ট (যেমন p1), যেখান থেকে sum() কল করা হয়েছে
p	অন্য পয়েন্ট অবজেক্ট (যেমন p2), যেটা self এর সাথে যোগ করতে হবে
self.x + p.x	$p1.x + p2.x \rightarrow$ x-কোঅর্ডিনেট যোগ করা হয়েছে
self.y + p.y	$p1.y + p2.y \rightarrow$ y-কোঅর্ডিনেট যোগ করা হয়েছে
Point(...)	নতুন পয়েন্ট তৈরি করে রিটার্ন করে

উদাহরণ সহ ব্যাখ্যা:

```
p1 = Point(3, 2) # p1.x = 3, p1.y = 2
p2 = Point(6, 3) # p2.x = 6, p2.y = 3

result = p1.sum(p2)
```

এখানে `p1.sum(p2)` মানে: "p1, তোমার সাথে p2 যোগ করো এবং নতুন একটি পয়েন্ট তৈরি করো।"

এরপর এই অংশ চলবে: `return Point(self.x + p.x, self.y + p.y)` অর্থাৎ, `return Point(3 + 6, 2 + 3)` `return Point(9, 5)`

অর্থাৎ, `result` এর মান হবে $\rightarrow x = 9, y = 5$

বাস্তব উদাহরণ (সহজ ভাবে চিন্তা করো):

ধরো p1 হলো: তুমি ৩ ধাপ সামনে এবং ২ ধাপ ডানে গেছো। p2 হলো: তুমি ৬ ধাপ সামনে এবং ৩ ধাপ ডানে গেছো। তাহলে `p1 + p2` মানে হলো: মোট ৯ ধাপ সামনে এবং ৫ ধাপ ডানে গেছো।

সংক্ষিপ্ত সারমর্ম:

`sum()` মেথড → দুই পয়েন্টের x ও y আলাদাভাবে যোগ করে একটি নতুন Point তৈরি করে।

কোড সহ ধাপে ধাপে বোঝা:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def sum(self, p):
        return Point((self.x + p.x), (self.y + p.y))

    def print_point(self):
        print(f"X is {self.x} and Y is {self.y}")

    def __add__(self, p):
        return Point((self.x + p.x), (self.y + p.y))

p1 = Point(3, 2)
p2 = Point(6, 3)

# p = p1.sum(p2)  # sum() মেথড ব্যবহার করে যোগ করা
p = p1 + p2       # + অপারেটর ওভারলোডিং এর মাধ্যমে যোগ করা
p.print_point()
```

ব্যাখ্যা:

কোডের উদ্দেশ্য: আমরা একটি `Point` নামের ক্লাস তৈরি করছি, যা ২D কোঅর্ডিনেট সিস্টেমের একটি পয়েন্ট প্রতিনিধিত্ব করে। এই ক্লাস দিয়ে আমরা: ১. পয়েন্ট তৈরি করতে পারি (x, y) ২. দুই পয়েন্ট যোগ করে নতুন পয়েন্ট তৈরি করতে পারি ৩. পয়েন্টের x ও y সুন্দরভাবে প্রিন্ট করতে পারি

ধাপে ধাপে ব্যাখ্যা:

☒ ধাপ ১: ক্লাস ডিফাইনেশন

```
class Point:
```

এটা দিয়ে আমরা `Point` নামের একটি ক্লাস তৈরি করলাম — যা হলো ব্লুপ্রিন্ট বা ডিজাইন, যেটা দিয়ে অনেক পয়েন্ট তৈরি করা যাবে।

☒ ধাপ ২: কনস্ট্রাক্টর (init method)

```
def __init__(self, x, y):
    self.x = x
```

```
self.y = y
```

- `__init__` হলো কনস্ট্রাক্টর, যা অবজেক্ট তৈরি হলে স্বয়ংক্রিয়ভাবে চালু হয়।
- এখানে দুইটি প্যারামিটার নেওয়া হয়েছে: `x` এবং `y`।
- `self.x = x` অর্থ হলো, এই অবজেক্টের জন্য `x` নামে ভেরিয়েবল তৈরি হলো এবং সেটাতে প্যারামিটার `x` এর মান সেট করা হলো।
- একইভাবে `self.y = y` সেট করা হলো।
- মনে রাখবে: `self` মানে হলো এই অবজেক্ট নিজেই।

☑ ধাপ ৩: `sum()` মেথড

```
def sum(self, p):
    return Point((self.x + p.x), (self.y + p.y))
```

- `self` হলো প্রথম পয়েন্ট (`p1`) যেখান থেকে মেথড কল করা হয়েছে।
- `p` হলো দ্বিতীয় পয়েন্ট (`p2`) যা যোগ করতে হবে।
- তারপর `x` এবং `y` আলাদাভাবে যোগ করে নতুন একটি `Point` অবজেক্ট তৈরি করে রিটার্ন করে।

☑ ধাপ ৪: `print_point()` মেথড

```
def print_point(self):
    print(f"X is {self.x} and Y is {self.y}")
```

- পয়েন্টের `x` ও `y` সুন্দরভাবে প্রিন্ট করার জন্য একটি মেথড।
- এখানে f-string ব্যবহার করে সরাসরি আউটপুট দেওয়া হয়েছে: `👉 X is 9 and Y is 5`

☑ ধাপ ৫: অবজেক্ট তৈরি

```
p1 = Point(3, 2)
p2 = Point(6, 3)
```

- এখানে দুটি পয়েন্ট অবজেক্ট তৈরি হলো: `p1.x = 3, p1.y = 2` `p2.x = 6, p2.y = 3`

☑ ধাপ ৬: পয়েন্ট যোগ করা

```
p = p1.sum(p2)
```

- `p1.sum(p2)` মানে: ১. `p1` এর `x` এবং `p2` এর `x` যোগ করা $\rightarrow 3 + 6 = 9$ ২. `p1` এর `y` এবং `p2` এর `y` যোগ করা $\rightarrow 2 + 3 = 5$

- এরপর নতুন পয়েন্ট p তৈরি হলো, যার x=9, y=5

☑ ধাপ ৭: আউটপুট প্রিন্ট করা

```
p.print_point()
```

- `print_point()` মেথড কল হলে আউটপুট হবে: **X is 9 and Y is 5**

টেবিল আকারে স্মারক:

অবজেক্ট / অংশ	ব্যাখ্যা
self	বর্তমান অবজেক্ট যেটার মাধ্যমে মেথড কল হয়েছে
init	অবজেক্ট তৈরি করার সময় স্বয়ংক্রিয়ভাবে চালু হয়
Point(...)	নতুন অবজেক্ট তৈরি করার জন্য ব্যবহার হয়
মেথড (method)	ক্লাসের ভিতরের ফাংশন
p1.sum(p2)	দুই পয়েন্ট যোগ করে নতুন পয়েন্ট তৈরি করে

🎁 বোনাস আইডিয়া:

তুমি চাইলে এখন নিচের ফিচারগুলো যোগ করে কোডটা আরও আকর্ষণীয় করতে পারো:

- `__str__()` → যাতে `print(p)` করলে সুন্দর আউটপুট দেখায়
- `__add__()` → যাতে `p1 + p2` করে পয়েন্ট যোগ করা যায় (যেমন কোডে করা হয়েছে)
- `distance_to()` → দুই পয়েন্টের মধ্যে দূরত্ব বের করার জন্য

যদি এই অংশগুলোর অনুবাদ বা বাকি অংশও দরকার হয়, জানাও!

অবশ্যই! নিচে Object-Oriented Programming (OOP) এর বাংলা অনুবাদ দিলাম, তোমার দেওয়া মূল উদাহরণ এবং ব্যাখ্যার সাথে:

***::: **অবজেক্ট-ওরিয়েন্টেড প্রোগ্রামিং (OOP)** হলো এমন একটি প্রোগ্রামিং পদ্ধতি যেখানে আমরা বাস্তব জীবনের চিন্তাভাবনাকে কোডে রূপান্তর করি। আমরা সবকিছুকে অবজেক্ট হিসেবে দেখি।

এই অবজেক্টগুলো তৈরি হয় একটি ক্লাস থেকে, যা হলো একটি ব্লুপ্রিন্ট বা ডিজাইন।

🔗 বাস্তব জীবনের উদাহরণ: ধরো তুমি একটি গাড়ির ডিজাইন করছো। সেই ডিজাইন হলো **ক্লাস**। এখন যদি তুমি সেই ডিজাইন থেকে আসল একটি গাড়ি বানাও — সেটাই হলো **অবজেক্ট**।

◦ ক্লাস = ডিজাইন / ব্লুপ্রিন্ট ◦ অবজেক্ট = সেই ডিজাইন থেকে তৈরি বাস্তব জিনিস

১. এনক্যাপসুলেশন (Encapsulation)

সংজ্ঞা: এনক্যাপসুলেশন হলো ডেটা এবং ফাংশনগুলোকে একসাথে একটি ক্লাসের ভিতরে রাখা যাতে বাইরের কোন প্রোগ্রাম সরাসরি ভিতরের ডেটা অ্যাক্সেস করতে না পারে। উদাহরণস্বরূপ, `__balance` নামক ভেরিয়েবল ক্লাসের বাইরের থেকে সরাসরি অ্যাক্সেস করা যায় না। এটিই হলো এনক্যাপসুলেশন। 📦

🧺 বাস্তব জীবনের উদাহরণ: রসের কার্টন বা বোতল

তুমি যখন রস খাও, তুমি শুধু বোতল খুলে রস খাও। কিভাবে রস তৈরি হয়েছে বা কত পানি মেশানো হয়েছে, সেটা জানার দরকার নেই।

১.১ বোতলের ভেতরের রস গোপন রাখা হয় (এনক্যাপসুলেটেড)। ১.২ তুমি শুধু দরকারি কাজ করো (রস খাওয়া)।

১.৩ ঠিক একইভাবে প্রোগ্রামিংয়ে আমরা ডেটা ও মেথডগুলো ক্লাসের ভিতরে রাখি এবং বাইরের দিক থেকে সরাসরি অ্যাক্সেস আটকাই।

২. ইনহেরিটেন্স (Inheritance)

সংজ্ঞা: ইনহেরিটেন্স মানে হলো একটি ক্লাস (চাইল্ড/ডেরাইভড ক্লাস) অন্য একটি ক্লাসের (প্যারেন্ট/বেস ক্লাস) বৈশিষ্ট্য এবং আচরণ পেয়ে যায়।

👨 বাস্তব জীবনের উদাহরণ: মানুষ এবং সন্তান

সন্তান তার পিতা-মাতার কিছু বৈশিষ্ট্য (যেমন চোখের রং, উচ্চতা, আচরণ) পায়। আর নিজস্ব কিছু গুণাবলী (যেমন সঙ্গীত প্রতিভা) যোগ করতে পারে।

২.১ যেমন সন্তান পিতামাতার গুণাবলী পায়, তেমনি প্রোগ্রামিং-এ একটি ক্লাস অন্য ক্লাস থেকে বৈশিষ্ট্য এবং মেথড পায়।

২.২ পাইথনে একটি ক্লাস অন্য একটি ক্লাস থেকে বৈশিষ্ট্য এবং মেথড পেতে পারে — এটিই ইনহেরিটেন্স।

৩. পলিমরফিজম (Polymorphism)

সংজ্ঞা: পলিমরফিজম মানে একই ফাংশন বা মেথডের নাম বিভিন্ন প্রসঙ্গে বিভিন্ন রকম আচরণ করা।

🐼 বাস্তব জীবনের উদাহরণ: "চলো" শব্দটি

- "চলো স্কুলে যাই।"
- "গাড়ি চলোচ্ছে।"
- "পাখি উড়ে গেল।"

একই "চলো" শব্দের বিভিন্ন অর্থ।

৩.১ এটিই হলো পলিমরফিজম — একই নাম, কিন্তু বিভিন্ন আচরণ।

৩.২ পাইথনে একই মেথড বা ফাংশন নাম বিভিন্ন ক্লাসে ভিন্ন ভাবে কাজ করতে পারে।

৪. অ্যাবস্ট্রাকশন (Abstraction)

সংজ্ঞা: অ্যাবস্ট্রাকশন মানে দরকারি তথ্যগুলো দেখানো এবং জটিল অভ্যন্তরীণ বিষয়গুলো লুকিয়ে রাখা।

পাইথনে অ্যাবস্ট্রাক্ট ক্লাস তৈরি করতে আমরা `abc` (Abstract Base Class) মডিউল ব্যবহার করি।

🚗 **বাস্তব জীবনের উদাহরণ:** গাড়ি চালানো

তুমি যখন গাড়ি চালাও,

- স্টিয়ারিং ধরে,
- ব্রেক চাপো,
- গিয়ার বদলাও,

কিন্তু তুমি জানো না (এবং জানার দরকার নেই) ইঞ্জিন কিভাবে কাজ করে, পিস্টন কিভাবে চলে ইত্যাদি।

৪.১ তুমি শুধু দরকারি কাজগুলো করো — জটিল বিষয়গুলো লুকানো থাকে।

৪.২ পাইথনে আমরা অ্যাবস্ট্রাক্ট ক্লাস দিয়ে শুধু দরকারি মেথডগুলো দেখাই, বাকিটা লুকিয়ে রাখি।

যদি আরো ব্যাখ্যা বা উদাহরণ দরকার হয়, বলো।